

Домашнее задание 5

Лиза Фоменко

Задание 1

Оцените сложность алгоритма Divide, приведенного на странице 19 книги Дасгупты.
Сначала выпишем алгоритм

```
0 def Divide(x: int (n bit), y: int >= 1 ) -> q, r:
1     if x==0:
2         return 0,0
3
4     q, r = Divide(x/2, y)
5     q = 2q
6     r = 2r
7     if x[-1] == 1:
8         r = r + 1
9     if r >= y:
10        r = r - y
11        q = q + 1
12    return q, r
```

Рассмотрим алгоритм построчно. за $\Theta(1)$ мы проверяем, равен ли x 0. Затем мы рекурсивно вызываем функцию *Divide* для $\lfloor \frac{x}{2} \rfloor$ и y . Заметим, что (как оговаривалось на лекции), деление числа на 2 реализовано за $O(1)$, так как мы сдвигаем число на один разряд вправо. Каждый новый вызов *Divide* мы сдвигаем x на еще один разряд вправо. Так как по условию x - n -битное число, то таких сдвигов будет ровно n . Значит, функция *Divide* будет рекурсивно вызвана n раз.

Далее оценим тело самой функции.

В строках 4 и 5 происходит умножение на 2 – то есть левый сдвиг за $O(1)$ каждое. Итого $O(1)$ за две строки.

В строке 7 мы проверяем число на нечетность: для этого нужно последний бит числа x , $x[-1]$, сравнить с 1. Это атомарная операция, выполняется за $O(1)$. При необходимости мы выполняем операцию сложения $r = r + 1$ за $O(n)$ в строке 8 (мне кажется все-таки за n , а не 1, потому что мы принимаем за $O(1)$ атомарные операции, а число r имеет не более n разрядов).

В строках 10-12 также выполняется одна проверка n -битных чисел (за $O(n)$ в атомарных операциях), а далее следуют две операции сложения и вычитания за $O(n)$ (так как y n -битовое число)

Тогда все тело функции *Divide* имеет асимптотику $O(n)$. Так как *Divide* вызывается n раз, то общее время работы алгоритма $O(n^2)$.

Задание 2

Предложите алгоритм возведения n -битовых чисел в n -битовую степень по n -битовому модулю, оцените его сложность.

Задание 3

Кто-то очень умный пришел и сказал, что умеет возводить числа из n бит в квадрат за линейное время. Докажите, что тогда можно перемножать и разные числа за O от максимума их длин.

Буду пользоваться тем, что $(a + b)^2 = a^2 + 2ab + b^2$.

Выразим отсюда $ab = \frac{(a+b)^2 - a^2 - b^2}{2}$

Теперь исследуем каждую операцию

- сложение $a + b$ - сложение двух n -битных чисел, работает за $O(n)$, где n - максимальная из длин чисел
- возведение в квадрат $(a + b)^2$ работает за $O(n)$ согласно кому-то оч умному
- возведение n -битных чисел a, b в квадрат a^2, b^2 тоже работает за $O(n)$, n - длина числа
- осталось два вычитания n -битных чисел за $O(n)$, где n - максимальная длина числе
- деление на 2 - это сдвиг вправо на разряд, за $O(1)$

Итого ab можно найти за $O(n)$, где n - максимальная из их длин

Задание 4

Докажите, что $\text{НОД}(a, b) = \text{НОД}(a - b, b)$

Пусть мы знаем, что $\text{НОД}(a, b) = k$ Тогда $a = kr, b = km$

Распишем $a - b = kr - km = k(r-m)$ Тогда мы ищем $\text{НОД}(k(r-m), km)$

Пусть $\text{НОД}(k(r-m), km) = l$, тогда по условию $l > k$ (мы видим, что k является делителем $k(r-m)$ и km , но по предположению не является их НОДом, тогда $k < l$). Это значит, что мы можем записать два выражения:

$$k(r-m) = ol = a - b; km = il = b$$

Из этого равенства мы можем найти a : $a = a - b + b = ol + il = l(o+i)$. Тогда мы получили, что $a = l(o+i)$ и $b = il$, то есть l является делителем a и делителем b , при этом $l > k$ по предположению, значит, k не может быть $\text{НОД}(a, b)$, что противоречит изначальному условию.

Тогда $l = k$ и $\text{НОД}(a, b) = \text{НОД}(a-b, b) = k$

Задание 5

На доске написан набор положительных целых чисел. За один ход можно взять любые два числа и вычесть из большего меньшее. Процесс останавливается, когда все числа становятся одинаковыми. Докажите, что этот процесс всегда остановится. Какие числа останутся в результате?

Докажем, что процесс остановится. При случайном выборе X_1, X_2 , пусть без ограничения общности $X_1 > X_2$. За наш шаг мы меняем $X_1 = X_1 - X_2$. Подозрительно похоже на предыдущий пункт, будем держать в уме, что $\text{НОД}(X_1, X_2)$ от этой операции не меняется. Кстати интересно, что мы выбирали X_1, X_2 без ограничения общности, тогда за все попарные НОД между числами не изменятся в ходе наших операций.

Докажем детерминированность: $X_1 = X_1 - X_2$, при этом X_1 остается положительным числом после манипуляции. Так как мы из большего положительного числа всегда вычитаем меньшее положительное число, то мы всегда будем получать положительные числа и никогда не дойдем до отрицательных, поэтому процесс остановится. Остановится он тогда, когда все числа будут одинаковыми и мы не сможем выбрать меньшее из них.

Тогда нужно понять, какое именно число (ну по факту это число будет повторяться n раз, n - размер исходной последовательности) останется. Тут можно воспользоваться тем, что НОД каждой пары чисел не изменяется после произведенной операции: тогда и НОД всего множества чисел не изменится. Тогда, оперируя двумя фактами, все числа равны и НОД множества не изменился с изначального, выводим, что в результате каждое число будет заменено на $\text{НОД}(X_1, X_2, \dots, X_n)$

Задание 6

Оцените асимптотику рекуррентны $T(n) = T(\alpha n) + T((1 - \alpha)n) + \Theta(n)$ для $\alpha \in (0, 1)$.

Эту задачу будем решать с помощью дерева рекурсии. Пусть (для удобства), без ограничения общности, $\alpha < 1 - \alpha$.

Докажем с помощью дерева рекурсии. Наше дерево бинарное, но не обязательно полное, его высота: от $\log_{\frac{1}{\alpha}} n$ (самый близкий к корню, мы его получим по ветке $\alpha n \rightarrow \alpha n \rightarrow \alpha n \dots$) до $\log_{\frac{1}{1-\alpha}} n$ (самый далекий, его получим по ветке $(1 - \alpha)n \rightarrow (1 - \alpha)n \rightarrow (1 - \alpha)n \dots$). Логика моего решения будет такая:

```

      n
    /   \
  an    (1-a)n    -> n
    /       \    -> an + (1-a)n = n
  a^2n   a(1-a)n a(1-a)n (1-a)^2n    ->
    /       \       /       \    -> a^2n + 2a(1-a)n + (1-a)^2n = n
.....
/ \ / \ / \ / \ / \ / \ / \ / \ / \ # level log{1/a} n
0(1)
.....
      / \ # level log{1/(1-a)} n
    0(1) 0(1)

```

Итак, а каждом уровне до $\log_{\frac{1}{\alpha}} n$ проводится ровно n операций, которые входят в $f(n)$. На уровнях с $\log_{\frac{1}{\alpha}} n$ до $\log_{\frac{1}{1-\alpha}} n$ листья выполняются за $O(1)$, а шагов из $f(n)$ выполняется не более ($\leq n$), так как рекурсия из листьев не вызывается. Тогда мы можем оценить число шагов алгоритма как: $n \log_{\frac{1}{\alpha}} n \leq T(n) \leq n \log_{\frac{1}{1-\alpha}} n$, а, как мы доказывали раньше (типа в 1 дз что ли), это означает, что $T(n) = \Theta(n \log n)$

(Это просто обобщение задачи из предыдущей домашки, 2пункт5, которая еще разбиралась на семинаре)

Задание 7

Оцените трудоемкость алгоритма, разбивающего задачу размера n на n подзадач размера $\frac{n}{2}$, используя для этого $\Theta(n)$ операций.

Попробуем записать так: $T(n) = nT(\frac{n}{2}) + \Theta(n)$

Нарисуем дерево рекурсии. Оно будет полным.

```

      n
    /   |   \
  n/2 n/2 n/2
 / | \ | \ | \ | \
n/4 n/4 n/4 n/4 n/4 n/4 n/4 n/4 n/4 n/4
.....
/ \ / \ / \ / \ / \ / \ / \ / \ #
0(1) 0(1) 0(1) 0(1) 0(1) 0(1) 0(1) 0(1) 0(1) 0(1)

```

Попробуем по порядку найти зависимость.

На первом уровне мы делим задачу размера n на n задач размера $n/2$, тратя на это n шагов. Тогда на первом уровне дерева будет находиться n узлов, каждый из них равен $\frac{n}{2}$.

На следующем вызове рекурсий каждый из n узлов первого уровня вызовет $\frac{n}{2}$ подзадач размера $\frac{n}{4}$, затрачивая на это $\frac{n}{2}$ операций каждый. Тогда: общие затраты = количество узлов * шаги единичного разбиения = $n * \frac{n}{2} = \frac{n^2}{2}$ шагов. При этом на втором уровне дерева будет $n * \frac{n}{2} = \frac{n^2}{2}$ узлов, каждый размера $\frac{n}{4}$

Какая закономерность? С каждым последующим уровнем размер узла уменьшается вдвое. Количество узлов при этом увеличивается в такое количество раз, равное размеру узлов на предыдущем уровне. Количество шагов на уровне равно количеству узлов умноженное на размер единичного узла. Тогда:

На k уровне: каждый узел имеет размер $\frac{n}{2^k}$ (пусть у нас корень - это 0 уровень). Тогда количество этих узлов составляет $n * \frac{n}{2} * \frac{n}{4} * \dots * \frac{n}{2^{k-1}} = \frac{n^k}{2^{0+1+2+3+\dots+k-1}} = \frac{n^k}{2^{\frac{k(k-1)}{2}}}$. И каждый узел будет выполнять разбиение за $\Theta(\frac{n}{2^k})$, то есть всего время разбиение уровня составит $n * \frac{n}{2} * \frac{n}{4} * \dots * \frac{n}{2^{k-1}} * \Theta(\frac{n}{2^k}) = \frac{n^k}{2^{\frac{k(k-1)}{2}}} \Theta(\frac{n}{2^k}) = \Theta(\frac{n^{k+1}}{2^{\frac{k(k+1)}{2}}})$

Теперь вспомним, что наше дерево полное. От корня n мы дойдем до узлов 1, каждый раз уменьшая узел вдвое, за $\log_2 n$.

Тогда время работы алгоритма $T(n)$ будет равно сумме делений каждого уровня. Найдем сумму: $\Theta(T(n)) = \sum_{k=0}^{\log_2 n} \Theta(\frac{n^{k+1}}{2^{\frac{k(k+1)}{2}}})$. Эта сумма будет определяться последним членом: $\frac{n^{\log_2 n + 1}}{\log_2 n (\log_2 n + 1)} = n^{\frac{\log_2 n}{\log 4}} = \Theta(n^{\log n})$

Ответ: $T(n) = \Theta(n^{\log n})$

такое же достигается по мастер-теореме: $a = n, b = 2, c = \log_2 n; n = O(n^{c-\varepsilon}) = O(n^{\log_2 n - \varepsilon})$ верно $\forall \varepsilon \Rightarrow T(n) = \Theta(n^c) = \Theta(n^{\log n})$. Оно кажется правильным, но оценка n через $O(n^{\log n})$... не знаю. странно